1. Tìm kiếm chi phí đều (UCS)

1.1. Mục tiêu

Phần này trình bày thuật toán tìm kiếm chi phí đều (Uniform-Cost Search, UCS) dùng để giải các màn Bloxorz. UCS tìm dãy hành động đưa khối từ trạng thái bắt đầu tới đích sao cho tổng chi phí các bước là nhỏ nhất. Khác với BFS chỉ đếm số bước, UCS dùng hàm chi phí `StepCost` dùng chung trong `solvers/common.py`, nên có thể ưu tiên đường đi ít rủi ro hơn dù dài hơn.

Mã nguồn nằm trong `solvers/UCS.py`, gồm lớp `UCSSolver` kế thừa lớp trừu tượng `Solver` và chính sách chi phí `StepCost`. Lớp này chỉ chứa thuật toán tìm kiếm; luật chơi lấy từ `core/game_rules.py`, chi phí lấy từ `solvers/common.py`.

1.2. Mô hình hóa bài toán tìm kiếm

- **Trạng thái:** một `GameState` gồm `BlockState` (vị trí ô mốc `(r, c)`, tư thế đứng / nằm ngang / nằm dọc, cờ đã tách, vị trí hai khối lập phương và khối đang điều khiển) cùng `bridge_mask`, mặt nạ bit trạng thái đóng/mở của từng cầu. Hai trạng thái cùng vị trí khối nhưng khác trạng thái cầu là hai trạng thái khác nhau.
- **Trạng thái bắt đầu:** `GameState(level.start, level.initial_bridge_mask)`.
- **Hành động:** `U`, `D`, `L`, `R` lăn khối theo bốn hướng. Hành động `SPACE` chỉ hợp lệ khi khối đã bị tách bởi công tắc tách, dùng để đổi khối lập phương đang điều khiển (mục 2.5 của đề bài).
- **Hàm chuyển trạng thái:** `get_successors(state, level)` trả về các cặp `(action, next_state)` hợp lệ. Phần lõi đã loại các nước làm khối rơi ra ngoài bản đồ, vào ô trống hoặc cầu đang đóng, và các nước đứng thẳng trên ô dễ vỡ; đồng thời xử lý công tắc, tách và ghép khối.
- **Kiểm tra đích:** `is_goal(state, level)` đúng khi khối chưa tách, đang đứng và nằm trên ô đích. Một khối lập phương đơn lẻ trên ô đích không được tính là thắng.
- **Chi phí bước:** `step_cost(state, action, next_state, level)`, luôn dương (mục tiếp theo).

Mỗi màn có số ô hữu hạn và số cầu hữu hạn, nên không gian trạng thái hữu hạn.

1.3. Yêu cầu đầu vào và môi trường chạy

Màn chơi được đọc từ các tệp JSON trong `levels/` bằng `levels.level_loader`. Thuật toán nhận một `Level` và một `GameState` thuộc màn đó, rồi trả `SolverResult` gồm đường đi, tổng chi phí, số bước và số nút mở rộng. Thời gian và bộ nhớ được bổ sung khi gọi qua `run_solver()`.

Chương trình được viết bằng Python; phần tìm kiếm dùng thư viện chuẩn `heapq` và `itertools`, không phụ thuộc trực tiếp vào Pygame. Giao diện dùng `pygame-ce`, còn bộ kiểm thử của dự án dùng `pytest`, theo `requirements.txt`. Phiên bản Python và hệ điều hành dùng để tạo số liệu được ghi riêng trong phần thực nghiệm, không suy ra từ máy đang xem báo cáo.

1.4. Hàm chi phí

1.4.1. Công thức

Gọi s là trạng thái trước, a là hành động và t là trạng thái sau. UCS dùng chi phí của lớp `StepCost`:

$$c(s, a, t) = e(s, a, t) + \sum_{x \in C(t)} p(x) + b(s, t).$$

Trong đó e là công sức của chính nước đi, $C(t)$ là tập ô mà khối chiếm sau khi đi, $p(x)$ là mức phạt rủi ro của ô x và $b(s, t)$ là mức phạt khi nước đi làm thay đổi trạng thái cầu.

Bảng 1: Công sức của từng loại nước đi.

Loại nước đi	Công sức e
Đứng → nằm (lật đổ)	1.0
Nằm → đứng (dựng khối lên)	1.5
Nằm → nằm (lăn theo cạnh dài)	1.2
Di chuyển một khối lập phương khi đã tách	0.8
`SPACE`: đổi khối lập phương đang điều khiển	0.3

Bảng 2: Mức phạt rủi ro.

Tình huống	Mức phạt
Mỗi ô dễ vỡ bị chiếm	+0.5
Mỗi ô cầu bị chiếm	+0.3
Mỗi ô bị chiếm có ô kề (4 hướng) là khoảng trống	+0.1
Nước đi làm thay đổi `bridge_mask`	+0.5

1.4.2. Lý do chọn các hệ số

Đề bài không quy định chi phí; nếu mọi nước đi có chi phí 1 thì UCS cho thứ tự tìm kiếm giống BFS. Các hệ số được chọn theo hai ý:

- **Công sức:** dựng khối lên phải nâng khối nên đắt nhất; lăn khi đang nằm tốn hơn lật đổ một chút; một khối lập phương chỉ nặng bằng nửa khối nên rẻ hơn; `SPACE` không di chuyển khối nhưng vẫn phải có chi phí dương để thuật toán không lặp `SPACE` vô hạn.
- **Rủi ro:** ô dễ vỡ, cầu và ô sát mép là những vị trí dễ dẫn tới thua; nước bấm công tắc thay đổi bản đồ nên được tính là thao tác tốn kém.

Mức phạt rủi ro nhỏ hơn hẳn công sức một nước đi. Vì vậy UCS vẫn ưu tiên đường ngắn, và chỉ chọn đường an toàn hơn khi các đường có độ dài gần nhau. Đây là mô hình định lượng do nhóm đặt ra, không phải phép đo vật lý.

1.4.3. Tính chất cần cho UCS

Mọi mức phạt đều không âm, nên với chính sách mặc định:

$$c(s, a, t) \geq \min(1.0, 1.5, 1.2, 0.8, 0.3) = 0.3 > 0.$$

Chi phí dương là điều kiện để UCS bảo đảm lời giải tối ưu (mục sau).

Ghi chú

Mức phạt của mỗi ô chỉ phụ thuộc bản đồ, nên `_precompute_cell_penalty` tính sẵn một bảng `ô → mức phạt` một lần ở đầu mỗi lần giải. Khi đó `step_cost` chỉ cộng công sức với vài phép tra bảng.

1.5. Thuật toán và cài đặt

1.5.1. Ý tưởng

UCS luôn mở rộng trạng thái có chi phí tích lũy nhỏ nhất:

$$g(s) = \sum_{i=1}^{k} c(s_{i-1}, a_i, s_i), \quad s_0 = \text{start}, s_k = s.$$

Hàng đợi ưu tiên sắp theo g. UCS là trường hợp riêng của A* với hàm ước lượng $h = 0$, và trùng với BFS khi mọi bước có cùng chi phí.

1.5.2. Mã giả

Thuật toán 1: Tìm kiếm chi phí đều trên đồ thị trạng thái Bloxorz

Đầu vào: Màn chơi, trạng thái bắt đầu, hàm chi phí dương `step_cost`.
Đầu ra: Dãy hành động có tổng chi phí nhỏ nhất, hoặc không có lời giải.

Tính sẵn mức phạt của các ô; đặt `best_cost[start] = 0`, `came_from[start] = None`.
Đưa `(0, counter, start)` vào hàng đợi ưu tiên `frontier`.
Khi `frontier` còn phần tử: lấy phần tử `(g, _, state)` có `g` nhỏ nhất.
Nếu `state` đã mở rộng hoặc `g > best_cost[state]`, bỏ qua (phần tử cũ).
Đánh dấu `state` đã mở rộng, tăng `expanded_nodes`.
Nếu `is_goal(state)`: dựng đường đi từ `came_from`, trả về chi phí `g`.
Với mỗi `(action, next)` trong `get_successors(state)` mà `next` chưa mở rộng:
    `new_g = g + step_cost(state, action, next)`.
    Nếu `new_g < best_cost[next]`: cập nhật `best_cost`, `came_from`, đưa `(new_g, counter, next)` vào `frontier`.
Nếu `frontier` cạn, trả về không có lời giải.

1.5.3. Cấu trúc dữ liệu

Bảng 3: Cấu trúc dữ liệu của `UCSSolver.solve`.

Biến	Vai trò
`frontier`	Đống cực tiểu (`heapq`) chứa `(g, counter, state)`.
`counter`	Bộ đếm tăng dần, phân giải hòa `g` theo thứ tự đưa vào, tránh so sánh hai `GameState`.
`best_cost`	Chi phí nhỏ nhất đã biết để tới mỗi trạng thái.
`came_from`	`trạng thái → (trạng thái trước, hành động)`, dùng để dựng lời giải.
`expanded`	Tập trạng thái đã mở rộng; mỗi trạng thái chỉ mở rộng một lần.

`GameState` là dataclass bất biến (`frozen=True`) nên dùng trực tiếp làm khóa của `dict` và phần tử của `set`. Khóa gồm cả `bridge_mask`, nên các cấu hình cầu khác nhau được phân biệt đúng.

1.5.4. Xử lý phần tử cũ và thời điểm kiểm tra đích

`heapq` không có thao tác giảm khóa. Khi tìm được đường rẻ hơn tới một trạng thái, thuật toán đưa thêm một bản mới vào hàng đợi; bản cũ có `g` lớn hơn `best_cost` sẽ bị bỏ qua khi được lấy ra và không được tính vào `expanded_nodes`.

Đích được kiểm tra khi trạng thái được **lấy ra** khỏi hàng đợi, không phải khi vừa sinh ra. Lúc lấy ra, mọi phần tử còn lại có g không nhỏ hơn, và mọi chi phí đều dương, nên không còn đường nào rẻ hơn tới trạng thái đó.

1.5.5. Tính đúng đắn và độ phức tạp

- **Đầy đủ:** không gian trạng thái hữu hạn và mỗi trạng thái chỉ mở rộng một lần, nên nếu có lời giải thì UCS tìm ra.
- **Tối ưu:** với $c \geq 0.3 > 0$, trạng thái được lấy ra lần đầu có g bằng chi phí đường đi nhỏ nhất (lập luận kiểu Dijkstra). Áp dụng cho trạng thái đích, lời giải trả về có tổng chi phí nhỏ nhất.
- **Thời gian:** $O(E \log E)$ với E là số cạnh đã xét; mỗi trạng thái có tối đa 5 trạng thái kế tiếp nên $E \leq 5N$, N là số trạng thái tới được.
- **Bộ nhớ:** $O(N + E)$ cho `best_cost`, `came_from`, `expanded` và hàng đợi (có thể chứa phần tử cũ).

1.5.6. Cài đặt và cách chạy

`UCSSolver` kế thừa `StepCost` để lấy hàm chi phí và `Solver` để có giao diện chung với các thuật toán khác. Đường đi được dựng bằng hàm dùng chung `reconstruct_path`. Kết quả trả về là `SolverResult`:

- Tìm được: `path`, `solution_length = len(path)`, `cost = g`, `success = True`.
- Không tìm được: `path = None`, `solution_length = -1`, `cost = inf`, `success = False`.

`search_time_ms` và `peak_memory_kb` do `run_solver` điền. Giao diện (`ui/dependencies.py`) trả về `UCSSolver` khi người dùng chọn UCS. Có thể chạy độc lập từ thư mục gốc dự án:

```
python3 -m solvers.UCS          # tất cả các màn
python3 -m solvers.UCS 1 8 10   # chỉ các màn 1, 8, 10
```

1.5.7. Đoạn mã nguồn cốt lõi

Đoạn mã dưới đây thể hiện bước cập nhật chi phí và đưa trạng thái vào hàng đợi trong `solvers/UCS.py`:

```
for action, next_state in get_successors(state, level):
    if next_state in expanded:
        continue
    new_g = g + self.step_cost(state, action, next_state, level)
    if new_g < best_cost.get(next_state, float("inf")):
        best_cost[next_state] = new_g
        came_from[next_state] = (state, action)
        heapq.heappush(frontier, (new_g, next(counter), next_state))
```

Điều kiện cập nhật dùng tổng chi phí, không dùng số hành động. Vì vậy thuật toán có thể chọn đường nhiều bước hơn nếu tổng chi phí thấp hơn.

1.6. Thực nghiệm

1.6.1. Cách bố trí

Chương trình `typst/collect_ucs_results.py` chạy hai cấu hình trên các màn `level_01` đến `level_10` trong thư mục `levels/`:

- **UCS:** `UCSSolver` với chi phí `StepCost` (bản được nộp).
- **UCS chi phí 1:** lớp con ghi đè `step_cost` trả về 1 cho mọi nước đi. Cấu hình này có thứ tự mở rộng như BFS, dùng làm đối chứng để thấy ảnh hưởng của hàm chi phí.

Mỗi cấu hình chạy thử một lần không ghi nhận, sau đó đo 5 lần qua `run_solver()` (cùng hàm đo với giao diện); bảng lấy trung vị thời gian và bộ nhớ. Thời gian đo bằng `perf_counter()` quanh `solve()`, bộ nhớ là mức cấp phát Python cao nhất theo `tracemalloc`. Vì `tracemalloc` theo dõi từng lần cấp phát, thời gian đo được cao hơn thời gian chạy thuần nhiều lần; ảnh hưởng này như nhau cho cả hai cấu hình. Môi trường: Python 3.11.7 trên Linux x86-64.

Mỗi lời giải được phát lại bằng `apply_move`: mọi hành động phải hợp lệ, trạng thái cuối phải thỏa `is_goal`, và tổng chi phí tính lại phải bằng chi phí thuật toán báo cáo. Kết quả của 5 lần đo phải trùng nhau. Tất cả các màn đều qua các phép kiểm tra này.

1.6.2. Kết quả

Bảng 4: Kết quả UCS với chi phí `StepCost` (trung vị 5 lần đo).

Màn	Bước	Chi phí	Số nút	Thời gian (ms)	Bộ nhớ (KB)
level_01	10	13.3	68	23.35	31
level_02	9	13.1	41	19.68	25
level_03	9	13.5	24	10.85	16
level_04	7	10.5	37	12.66	20
level_05	12	19.3	65	19.77	30
level_06	12	18.3	42	17.4	23
level_07	12	17.6	36	14.73	20
level_08	9	12.2	1444	572.6	939
level_09	11	15.3	3354	1520.54	1923
level_10	18	24.5	3925	1809.89	2032

Bảng 5: So sánh với cấu hình chi phí 1. Cột "chi phí đường chi phí 1" là chi phí `StepCost` của lời giải mà cấu hình chi phí 1 tìm được.

Màn	Bước UCS	Bước chi phí 1	Chi phí UCS	Chi phí đường chi phí 1	Nút UCS	Nút chi phí 1
level_01	10	10	13.3	13.3	68	65
level_02	9	9	13.1	13.2	41	44
level_03	9	9	13.5	13.5	24	25
level_04	7	7	10.5	10.5	37	32
level_05	12	12	19.3	19.3	65	62
level_06	12	12	18.3	18.3	42	43
level_07	12	12	17.6	17.6	36	37
level_08	9	9	12.2	12.2	1444	511
level_09	11	11	15.3	15.3	3354	1261
level_10	18	17	24.5	25.2	3925	3134

Số nút là số trạng thái được lấy khỏi hàng đợi và mở rộng, kể cả trạng thái đích; không tính phần tử cũ bị bỏ qua.

1.6.3. Nhận xét

- **UCS giải được cả 10 màn.** Lời giải được kiểm chứng bằng luật chơi và có chi phí nhỏ nhất theo `StepCost`.
- **Hàm chi phí thay đổi lời giải.** Ở `level_10`, UCS chọn đường 18 bước với chi phí 24.5, trong khi đường ngắn nhất 17 bước có chi phí 25.2. Ở `level_02`, hai cấu hình cùng 9 bước nhưng UCS chọn đường có chi phí 13.1 thay vì 13.2. Ở các màn còn lại, đường ngắn nhất cũng là đường rẻ nhất.
- **Các màn có công tắc tách tốn kém nhất.** `level_08`–`level_10` mở rộng hàng nghìn nút, gấp hàng chục lần các màn khác: sau khi tách, vị trí hai khối lập phương cùng khối đang điều khiển làm số trạng thái tăng mạnh.

- **Chi phí không đều làm UCS mở rộng nhiều nút hơn.** Trên ba màn có công tắc tách, UCS mở rộng khoảng 1.25–2.8 lần số nút của cấu hình chi phí 1. Nước đi của khối lập phương (0.8) và `SPACE` (0.3) rẻ hơn nước đi của khối nguyên, nên UCS khám phá rất nhiều tổ hợp khối lập phương trước khi xét đường dài hơn của khối nguyên. Đây là cái giá của việc tối ưu theo chi phí thay vì số bước. Trên các màn không có công tắc tách, số nút của hai cấu hình gần như nhau.

Hạn chế

UCS không biết đích nằm ở đâu nên lan đều theo mọi hướng. Để giảm số nút cần một hàm ước lượng chấp nhận được, tức là chuyển sang A*. Thuật toán cũng chưa có giới hạn thời gian hay số nút; trong giao diện, việc hủy được thực hiện bằng cách dừng tiến trình tìm kiếm.

1.7. Đối chiếu các phần thực hiện

Bảng 6: Đối chiếu nội dung báo cáo với phần cài đặt UCS.

Nội dung	Cách thực hiện và đối chiếu
Mô hình hóa bài toán	Dùng trạng thái, hàm chuyển và kiểm tra đích của phần lõi.
Thiết kế chi phí	Dùng `StepCost` chung, có chi phí chuyển động và mức phạt theo bản đồ.
Cài đặt tìm kiếm	Hàng đợi theo chi phí tích lũy, loại phần tử cũ, kiểm tra đích khi lấy ra.
Dựng lời giải	Dùng bảng cha và `reconstruct_path`; trả đúng `SolverResult`.
Kiểm chứng kết quả	Chương trình thu thập phát lại đường đi, kiểm tra đích và cộng lại chi phí.
Đo hiệu năng	Đo lặp qua `run_solver()`, tổng hợp trung vị từ các lần chạy.

Bảng này mô tả cách thực hiện, không tự quy đổi thành tỷ lệ hoàn thành. Phần đóng góp cá nhân và đánh giá mức độ hoàn thành cần được nhóm xác nhận.

1.8. Kết luận

UCS tìm lời giải có tổng chi phí nhỏ nhất theo quy tắc `StepCost` trên đồ thị trạng thái do phần lõi cung cấp. Số liệu ghi trong tệp thực nghiệm cho thấy mục tiêu này có thể tạo lời giải khác với tối thiểu số bước, đặc biệt ở màn có tách khối. Đổi lại, thuật toán có thể phải mở rộng nhiều trạng thái vì không sử dụng thông tin ước lượng khoảng cách tới đích.